OpsReplay

CS5224 Final Report

Learn production failures by investigating them.

Group 26

Lee Hao Zhe, Neeraj Kumbar, Neo Qi Hao, Poh Yu Wen, See Yang Zhi, Teo Kai Xiang

Summary. OpsReplay is a Software-as-a-Service for practising production incident response. Each learner gets a dedicated, deliberately broken cloud environment running real services, investigates it with ordinary tools in a browser terminal, repairs it, and then reviews an evidence-based debrief and a synchronised playback of the attempt. The prototype runs on AWS (CloudFront, Cognito, API Gateway, Lambda, DynamoDB, S3, ECS Fargate, EventBridge) and was deployed and tested end to end in bounded campaigns. Code: `https://github.com/ehz0ah/OpsReplay`.

1 Motivation and Business Case

Theme: cloud-based production-engineering education.

1.1 Problem Statement

Students and junior engineers rarely get to practise responding to a production failure before they are responsible for a live service. Postmortems explain what happened, but reading one does not require the reader to choose which evidence matters, form a hypothesis under uncertainty, or test a recovery decision whose consequences can make things worse. In real incidents, an attempted fix can provide temporary relief, cause a new outage, or delay recovery. The skill we target is therefore *diagnosis plus judgement*: finding the cause from incomplete signals and recovering safely. Google's *Wheel of Misfortune* trains this through facilitated team drills that re-enact past incidents [1], but such drills need a facilitator, a team and preparation time.

1.2 Purpose, Target Users and Value

OpsReplay turns this practice into an individual, on-demand web service. Its primary users are computer science students and junior software, DevOps, platform and site reliability engineers; a secondary market is course instructors and bootcamps who want a ready-made lab. It has three modes:

- **Challenges** (core): a live, isolated environment with one planted misconfiguration. Learners use a root shell and real tools (`curl`, `dig`, `ss`, logs, config files). Traffic keeps flowing, so failures accumulate while the fault remains, and careless actions cause real outages. An independent monitor decides recovery.
- **Learn**: curated, team-written summaries of documented incidents with lessons and links to official reports.
- **Code Review**: learners flag risky lines in a diff and compare them with reviewed reference findings.

After a Challenge, learners get a debrief (root cause, causal chain, evidence they found or missed, harmful actions, recommended recovery) and can replay their terminal session beside the recorded metrics, or retry in a fresh environment without overwriting the first result.

1.3 Comparison with Existing Services

Service	What it offers	Gap OpsReplay addresses
Google SRE drills [1]	Facilitated team re-enactments of past incidents	Needs a facilitator and team; no self-service environment
SadServers [2]	Real Linux troubleshooting servers with a pass/fail check	No live traffic or measured user impact, no causal debrief or session playback
SimIncident [3]	Simulated incident-management and coordination exercises	Focuses on coordination rather than hands-on diagnosis of real services
Postmortem reading	Free, rich narratives	Passive; no decisions or consequences
OpsReplay	Individual real-service investigation with continuous traffic, independent recovery checks, measured user impact, an evidence-based debrief, synchronised playback, fresh retries, and Learn and Code Review in one product.	

Interactive troubleshooting labs already exist, so our claim is not that the category is new. The differentiator is the closed loop of *investigate* → *act* → *observe measured consequence* → *debrief* → *replay*.

1.4 Changes since the Preliminary Report

An early version of our design simulated incidents with a deterministic state machine and authored metric rules. We replaced it with real services in per-attempt containers because authored rules could only reward anticipated actions, whereas real systems respond correctly to any valid fix or mistake. This raised the engineering cost (container isolation, recording, cleanup) but made the learning experience authentic. The scope was also reduced to two published Challenges so that the full workflow could be hardened and tested on AWS; the third (connection exhaustion, Hard) remains a draft.

2 Business Model

2.1 Freemium Offer

- **Free**: a complete sample of every mode, including the Easy Challenge with full debrief, playback and fresh retries, so a learner experiences the core value before paying. Sessions last 20 minutes.
- **Pro** (proposed S$12/month): the expanded library (Medium/Hard Challenges, Pro lessons and reviews) and 30-minute sessions. Entitlements are enforced on the server, not just hidden in the interface.
- **Institutional**: Pro seats licensed to a class or cohort (future).

The price is benchmarked against SadServers (US$9–11/month for Pro tiers [2]); it is a hypothesis that still needs willingness-to-pay testing. The prototype implements accounts, plans and a time-limited "demo Pro" grant, but deliberately collects no payment.

2.2 Unit Economics

Each Challenge attempt runs its own Fargate task (1 vCPU, 2 GiB), costing US$0.0616 per hour at Singapore list prices [4]; in our hosted tests an attempt averaged about 11 minutes from image pull to stop, or about US$0.011 of compute. An assistant reply cost about US$0.0004 in tokens. Variable cost is therefore small; the dominant cost at small scale is *idle shared infrastructure* (Gateway task, internal load balancer, VPC interface endpoints), about US$97.8 (S$132) per month [5, 6].

Assumptions (planning FX S$1.35/US$1): a Pro user consumes 6 Challenge-hours, US$0.30 of assistant use and US$0.15 of API/storage per month; each subscriber subsidises five active Free users (0.5 hours each); future card fees are 4.1% + S$0.50 [7]. Variable cost per subscriber is then S$1.82 and contribution S$9.19. Monthly fixed cost is S$972 (S$132 shared infrastructure, S$40 other services, S$600 content and support, S$200 acquisition).

Scenario	Contribution / subscriber	Break-even subscribers
Base case, S$12	S$9.19	106
Heavy use: 60 Challenge-hours / subscriber	S$4.70	208
20 Free users per subscriber	S$7.04	138
US$3 assistant spend / subscriber	S$5.54	176
Price S$8 / S$16	S$5.35 / S$13.02	182 / 75

Infrastructure alone breaks even at 19 subscribers. The main commercial risk is heavy use: unlimited runtime cannot be offered, so a launch needs fair-use limits on monthly Challenge hours.

2.3 Cloud versus On-premise Hosting

Equivalent service. A fair comparison must host the *same* service, not merely run the three container images on a server. An on-premise OpsReplay would need: HTTPS ingress and static hosting; an identity service with verified email; a transactional database with conditional writes (for duplicate-safe requests); a private object store with backups for recordings; an isolated container scheduler with per-attempt networking and confirmed cleanup; durable timers and retry workers; and logging, alerting and secrets management. The code calls AWS SDKs directly, so a port also needs adapter work.

Model. We model one small server sized for two simultaneous attempts plus the platform, against H total Challenge-hours per month (all monetary values S$/month, before GST):

$$C_{\text{cloud}} = \underbrace{132.07}_{\text{shared infra}} + 0.0832H + \underbrace{40}_{\text{API,storage,logs}} + \underbrace{60}_{\text{2 h ops}},$$

$$C_{\text{on-prem}} = \underbrace{4000/36}_{\text{server, 3 yr}} + \underbrace{29.22}_{\text{100 W}\times 1.4\ @\ \text{S\$0.2859/kWh}} + \underbrace{100}_{\text{network,backup,maint.}} + \underbrace{180}_{\text{6 h ops}} + \underbrace{50}_{\text{port work/36}}.$$

Electricity uses the SP Group Q4 2026 tariff [8]; the S$4,000 server is a sensitivity input informed by local retail listings for entry servers (S$3,777–4,956) [9], not a quote. Operator time is valued at S$30/hour.

Challenge-hours / month	Cloud	On-premise (base)	On-premise (equal ops, no port)
50	S$236	S$470	S$300
400	S$265	S$470	S$300
1200	S$332	S$470	S$300

Justification. For OpsReplay's demand profile, cloud is the better choice, but not because it is unconditionally cheaper:

- *Bursty demand.* Usage arrives as class-sized bursts around assignments, not a steady load. Fargate scales to many parallel attempts and costs nothing when idle; a server sized for two attempts cannot absorb a 30-person class, and one sized for the peak sits idle most of the month.
- *Isolation.* Learners get root shells. Fargate gives each attempt its own kernel boundary, private networking and no cloud credentials; a shared Docker host isolates root users less strongly.
- *Availability and operations.* DynamoDB and S3 are multi-AZ managed services; a single non-redundant server is a single point of failure, and backups are not failover.

On-premise becomes competitive when an institution already owns qualified hardware and staff: with equal operator effort and no port work, it is cheaper above roughly 821 Challenge-hours per month. In the base case the crossover (about 2,864 hours) exceeds the two-attempt server's capacity (1,460 hours), so it is not reachable. The model's assumptions (labour, server price, power draw) are not measurements and are the main sources of uncertainty.

3 Architecture and Implementation

3.1 User Journey

A learner browses the catalogue (Figure 1), reads a public briefing that lists the services and where their logs and configuration live (without revealing the fault), signs in and starts the attempt. The workspace shows an alert, a browser terminal and a live telemetry panel (request rate, errors, latency). A notice explains that leaving the page does not pause the incident. Once the monitor confirms sustained recovery, or the time limit passes, the attempt ends and the learner sees the debrief, playback and a retry option. History keeps every saved result.

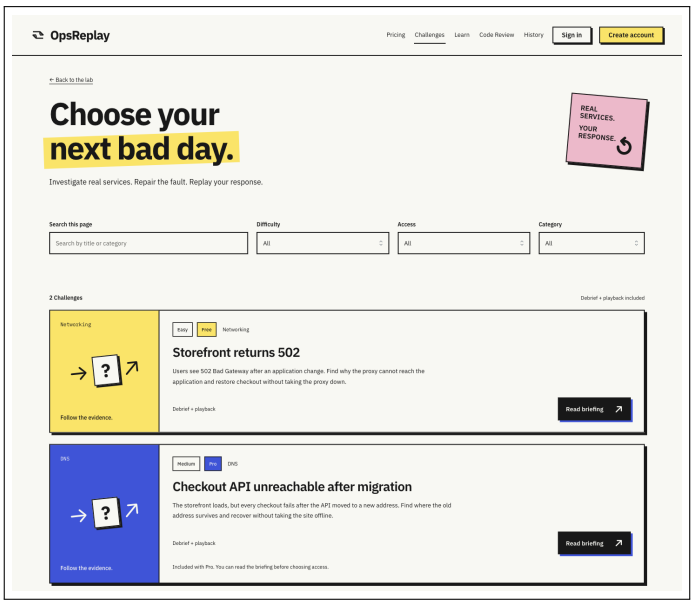


Figure 1: Challenge catalogue in the deployed web app (desktop width). Difficulty, access tier and category filters come from server metadata.

TODO (team): Optional but recommended: add one screenshot of the Challenge workspace (terminal + telemetry) and one of the debrief/playback page from the demo deployment. Keep the report within 10 pages.

3.2 Challenges

Each Challenge is a versioned container image with a small service stack under a process supervisor, plus one planted misconfiguration, described by a JSON manifest (traffic journeys, validators, traps, hints, debrief content).

Challenge	Planted fault	Why action order matters
Storefront returns 502 (Easy, Free)	nginx `proxy_pass` targets port 8081, but the app now listens on 8080	Editing without `nginx -t` risks a syntax error; a reload fails safely, but a restart takes the proxy down
Checkout API unreachable after migration (Medium, Pro)	CoreDNS still maps `api.internal` to a retired address that nginx cached at startup	Fixing DNS alone leaves checkout at 502 until nginx reloads; restarting nginx while the zone is broken takes the whole site offline
Connection exhaustion (Hard, draft)	Missing index plus pool size near PostgreSQL's limit	Adding workers exhausts connections; the index fixes the cause

Recovery is decided by validators, e.g. the checkout journey must succeed continuously for 60 seconds. There is no fixed action list: any valid fix passes, and traps are detected from real effects (an outage observed by the monitor shortly after a risky command), not by matching expected commands.

3.3 AWS Deployment

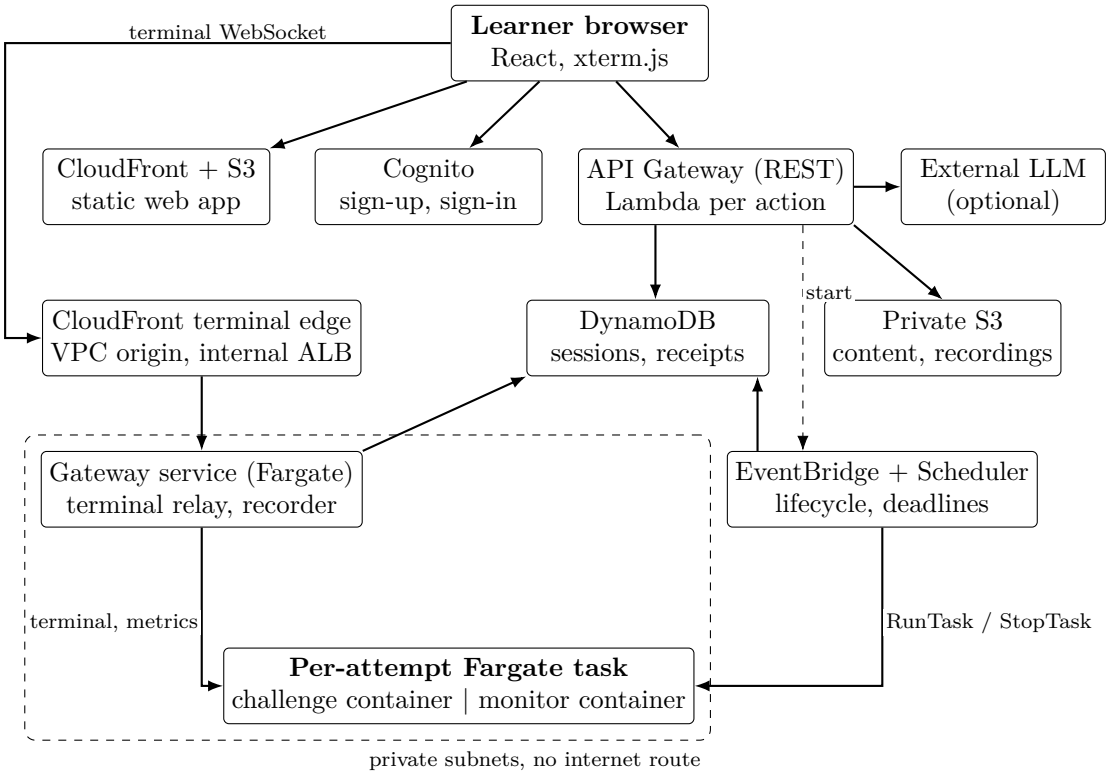


Figure 2: Deployed architecture (Region ap-southeast-1; terminal edge protection in us-east-1). The Gateway is the only component that talks to attempt tasks. CloudWatch logs, metrics and alarms cover all components (not drawn).

Figure 2 shows the deployment, defined in AWS CDK (TypeScript). **Web and identity:** the React single-page app is served from private S3 through CloudFront; Cognito handles verified-email sign-up and PKCE sign-in. **API:** a Regional API Gateway REST API routes to one Lambda function per action (start, end, ticket, debrief, playback, review, assistant, etc.), each with its own role, timeout and concurrency limit. REST was chosen over the cheaper HTTP API because it supports response streaming for assistant replies. **State:** DynamoDB stores sessions, request receipts, timelines, review submissions and conversation turns, all read by key; large recordings go to private S3. **Environments:** starting an attempt calls ECS `RunTask` with a task definition pinned to image digests. The task holds the challenge container (services plus a private terminal server) and a separate monitor container (traffic generator, metrics, validators, file captures). **Terminal path:** browser WebSockets enter a CloudFront distribution with WAF, reach an internal load balancer through a VPC origin, and land on the Gateway service, which validates a short-lived ticket, proxies the terminal and records everything. **Lifecycle:** ECS events and EventBridge Scheduler drive readiness, deadlines, abandoned-session sweeps and reconciliation.

3.4 Session Lifecycle and Reliability

An attempt moves through *admitted* → *provisioning* → *ready* → *outcome fixed* → *recording drained* → *task stopped* → *result saved*. Key design choices:

- **Duplicate-safe requests.** Before any side effect, the start handler saves, in one DynamoDB transaction, a request receipt, a one-active-attempt lock, the session record and the exact launch arguments. A retried or reloaded request reuses the same receipt and ECS launch identity, so a lost response never launches a second task or records a second result.
- **Single outcome.** The first valid outcome (recovered, time limit, learner ended, abandoned) fixes one cutoff; later end requests return it.
- **Stale-worker safety.** Lifecycle events are treated as hints: workers re-read state, and conditional

writes reject late timers, old generations and overlapping workers.
- **Confirmed cleanup.** The result is finalised only after the task stop and schedule deletion are confirmed, so no environment is left running.
- **Terminal resilience.** The shell survives browser disconnects; a reconnect resumes it with a newer connection generation, and if the Gateway is replaced, the new Gateway resumes recording.

3.5 Recording, Debrief, Scoring and Playback

The Gateway records terminal bytes, timestamped monitor measurements and watched-file captures *outside* the learner's writable environment, as checksummed pages in S3. Terminal output is never trusted to decide recovery; only the monitor does. The debrief is derived deterministically from the pinned Challenge manifest and the recorded evidence: root cause, causal chain, key evidence observed or missed, harmful actions with the outages that followed, and the recommended recovery. We deliberately avoided over-claiming: mentioning a file path is not counted as reading it, and a command preceding an outage is shown as "possibly harmful", not proven causal. Instead of an opaque weighted score, the result reports its components: time to recovery, failed user requests, command count, number and duration of self-inflicted outages, and hint/assistant use. Incomplete recordings get no score. Playback replays the terminal recording in sync with the telemetry, with markers for commands and key evidence; retries run in a fresh task and never alter the first result.

3.6 Learn and Code Review

Learn entries are Markdown with metadata; Free lessons are built into static assets, while Pro lessons stay in private S3 and are returned only after an entitlement check. Code Review shows source context with selectable lines and inline comments. Drafts survive refresh. Submission matches flagged lines against authored reference ranges deterministically and reports found and missed issues with explanations; written concerns are shown beside the reference but not graded.

3.7 Optional AI Assistant

In a Challenge, the assistant explains recent evidence and may propose one shell command through a typed `propose_command` tool. Its context is limited to public service information, completed commands and output, visible metrics, released hints and recent turns; the planted fault, validators and unreleased hints never enter the prompt. A proposal runs only after the learner confirms the exact text, the Gateway checks ownership and expiry, and the shell input buffer is empty; disruptive commands are flagged with a caution. The assistant never owns state, recovery or scoring, its use is recorded, and every Challenge can be completed with it disabled or failing. Replies stream through the REST API, and each turn is guarded by a request receipt so a retry never calls the provider twice.

3.8 Security and Isolation

Challenge tasks have no IAM role (a root shell exposes no AWS credentials), run in private subnets with no internet route, and accept traffic only from the Gateway. Every read and mutation checks identity, ownership and entitlement; public API responses use explicit projections defined in JSON Schema/OpenAPI contracts. Images are pinned by digest and must pass an Amazon ECR vulnerability scan gate before any deployment is activated.

4 Evaluation

Each result below is tied to a source revision, workload and environment; all hosted tests ran in `ap-southeast-1` in bounded, time-boxed deployments that were torn down afterwards.

4.1 Scenario Validity and Repeatability

Automated image suites check, for each Challenge, that the fault is present at start, validators fail before the reference fix and pass after it, and each trap produces the stated outage. Twenty repeats of the Challenge suite and twenty of the Monitor suite per incident (80 suite runs) all passed with

unchanged image IDs. The repository has 160+ test files across the API, Gateway, Monitor, content, web and infrastructure, run by CI (nine jobs) on every pull request.

4.2 Hosted Functional and Reliability Tests

Two hosted campaigns (29 October and 1 November) exercised the full product with test learners. The 1 November campaign ran six attempts, at most two concurrent, with one Gateway.

Test	Result
Sign-up, access, ownership	Verified-email sign-up and PKCE sign-in passed; guests got 401; Free access to the Pro Challenge got 403 without creating an attempt; another learner's session, debrief, playback, ticket and review requests got 404.
Both Challenges	Upstream: a validated reload restored checkout for 60 s. DNS: DNS-only repair left checkout at 502 and a validated reload recovered it; restarting nginx with a broken zone caused the expected outage.
Lost responses	Discarding the start (and review-submit) response and replaying recovered the same attempt and single task (the same submission).
Disconnects	An idle socket survived 610 s; an expired token returned 401 and a fresh sign-in resumed the same shell; killing the Gateway task let ECS replace it and the same attempt resumed and resolved.
Outcomes	Duplicate end requests kept one outcome; a real 30-minute deadline ended the attempt as *time limit*; a missed heartbeat ended one as *abandoned*.
Results	All six recordings finalised; all 41 exported objects matched their SHA-256 checksums; debrief, playback seeking and History were checked in the browser.
Isolation	No credentials in the task environment; EC2 metadata and the internet were unreachable (configuration check, not a penetration test).
Cleanup	All temporary stacks and images removed; durable data preserved.

The campaign also surfaced four reconciliation errors (mis-filtered Gateway service events and an outcome read race), which were fixed afterwards with regression tests.

4.3 Performance

A read-load test sent 450 authenticated GET requests (one worker: 150; two learners: 300) with two-seconds think time; all passed HTTP, schema and ownership checks, with no Lambda errors or throttles.

Endpoint	1 worker p50 / p95 (ms)	2 workers p50 / p95 (ms)
Catalogue	75 / 249	60 / 231
Account	66 / 221	54 / 219
Session	71 / 190	48 / 208

We did not reach the planned 10/50/100-session load tests: running many Fargate attempts was constrained by the course account's budget and quotas, so provisioning time and terminal latency at class scale remain unmeasured.

4.4 AI Assistant

Six synthetic Challenge situations were each run three times against the production adapter (19 attempts: 18 completed, 1 failed). Completed replies had p50/p95 latency of 4.6/6.0 s. Rubric review found a relevant command proposal in 18/18 replies, accurate teaching in 17/18, at least one unsupported factual claim in 4/18, and no destructive proposal. Two full deployed streams completed in 8.6 s and 6.6 s; the confirmed proposal executed exactly once even when the confirmation was repeated. Total token cost for all 24 provider calls was about US$0.01. Prompt changes followed to reduce unsupported claims.

4.5 Security Gate

On 1 November the Gateway and Monitor images had no scan findings; each Challenge image had 1 HIGH (glibc, accepted for this deployment model), 3 MEDIUM and 3 LOW. A 4 November rebuild found three new SQLite HIGH findings, so the gate correctly blocked activation; the team rebuilt SQLite from a fixed source (merged), and fresh registry qualification is the next step before redeploying.

4.6 Measured Cloud Cost

The six 1 November attempts used 4,010 Challenge-task seconds and 5,688 Gateway seconds: an estimated US$0.117 of Fargate compute at list prices, consistent with AWS Cost Explorer's account data (US$0.112). These figures exclude shared networking, which dominates at this scale (Section 2.3).

4.7 User Evaluation

Protocol. A formative pilot with 5–10 students or junior engineers compares the full interactive workflow (8 minutes investigating and repairing plus 4 minutes of debrief and playback) with a static written explanation of the other incident, with equal study time and equivalent causal information. Four assignment sequences counterbalance condition and order; the assistant is off. Before and after each condition, participants answer diagnosis and recovery questions scored with a common 0–8 rubric, then a transfer question on a new case. Short tasks cover Learn, Code Review and saved results, followed by the System Usability Scale (SUS) [10] and a brief interview. Participants use pseudonyms; consent and raw data are kept outside the repository.

TODO (team): Insert pilot results here: number recruited/completed and their experience; a table of pre/post rubric scores per participant by condition and incident (interactive vs. static; Upstream vs. DNS); mean paired difference; transfer scores; completion and self-inflicted outages; mean and range of SUS; and the top 3–5 usability problems from interviews with the changes made. Do not include synthetic or invented data. If the pilot cannot run before submission, replace this box with an honest statement and the informal feedback actually collected.

Limitations. The Upstream Challenge is Easy and DNS is Medium, so results are reported per incident; counterbalancing does not remove difficulty differences or carryover. A pilot of this size can guide improvements but cannot establish broad learning effectiveness, retention, or the separate effect of playback.

5 Conclusion and Future Work

OpsReplay delivers a working SaaS in which learners practise incident response on real, isolated services and learn from an evidence-based debrief and synchronised playback. The prototype demonstrates the full loop on AWS with duplicate-safe lifecycle handling, confirmed cleanup, entitlement checks and an optional, bounded AI assistant. Cloud hosting suits the bursty, isolation-heavy workload, although idle shared infrastructure dominates cost at small scale.

Future work: (1) publish more Challenges, starting with connection exhaustion, and a Pro Code Review set; (2) measure provisioning time and terminal latency at class scale and add a warm pool before scheduled classes; (3) reduce idle cost, for example by starting the Gateway and endpoints only around class windows; (4) instructor features: cohorts, assignments and progress dashboards for the institutional tier; (5) real payments and fair-use limits; and (6) a larger learner study of learning transfer and retention.

6 Code Explanation

Repository: `https://github.com/ehz0ah/OpsReplay` (public until marks and feedback are released). It is an npm workspace in TypeScript (Node 22), with Python for the in-container terminal server.

Path	Contents
`apps/web/`	React + Mantine single-page app: catalogue, briefing, xterm.js workspace, telemetry, debrief, playback, Learn, Code Review, History, account.
`apps/api/src/`	One folder per Lambda action (e.g. `start-session`, `end-session`, `create-terminal-ticket`, `get-debrief`, `assistant`, `submit-review`) plus lifecycle handlers (`session-lifecycle`, `sweep-sessions`, `reconcile-session`).
`apps/gateway/`	Fargate Gateway: ticket admission, browser terminal relay, terminal and monitor recording, proposal delivery.
`apps/monitor/`	Monitor container: traffic, measurements, validators, sustained-recovery checks and file captures.
`packages/session/`	Shared lifecycle, recording and deterministic debrief logic (`finalise.ts`, `debrief.ts`).
`packages/contracts/`	JSON Schemas, OpenAPI spec and generated public types.
`content/`	Challenge manifests and Docker images, Learn lessons, Code Review exercises.
`infra/`	AWS CDK stacks: durable data, browser hosting, session API, environments, terminal edge, deletion.
`scripts/evaluation/`	Scenario repeats, load tests and assistant evaluation.
`docs/`	Architecture, API, data model, design decisions, runbooks and test checkpoints.

Running locally: `npm ci`, then `npm run quality` (lint, format, type checks), `npm run check` and the component suites (`api:test`, `gateway:test`, `web:test`, `content:test`, `infra:test`). Docker is needed for the Challenge image tests (`challenge:build`, `challenge:test`) and the DynamoDB Local integration tests; `npm run web:dev` starts the web app. Deployment uses `npm run infra:cdk` with the parameters described in `infra/README.md` and `docs/runbooks/`; runtime stacks are deployed only for bounded test windows and torn down afterwards.

Assumptions. Prices are AWS list prices for Singapore before credits and tax; FX is S$1.35/US$1; usage, conversion and labour figures in Section 2 are planning assumptions, not measurements. The hosted tests were bounded acceptance tests, not production capacity or security certification.

References

[1] Google, "Postmortem Culture: Learning from Failure," *The Site Reliability Workbook*. `https://sre.google/workbook/postmortem-culture/`.

[2] SadServers, "FAQ" and "Pricing." `https://sadservers.com/pricing`.

[3] SimIncident. `https://simincident.com/`.

[4] AWS, "AWS Fargate Pricing." `https://aws.amazon.com/fargate/pricing/`.

[5] AWS, "Amazon VPC Pricing." `https://aws.amazon.com/vpc/pricing/`.

[6] AWS, "Elastic Load Balancing Pricing." `https://aws.amazon.com/elasticloadbalancing/pricing/`.

[7] Stripe Singapore, "Pricing." `https://stripe.com/en-sg/pricing`.

[8] SP Group, "Tariff Information," Q4 2026. `https://www.spgroup.com.sg/our-services/utilities/tariff-information`.

[9] Dell Singapore, "PowerEdge T160," and Sim Lim Square, "Dell PowerEdge T350," retail listings read 31 Oct–4 Nov 2026.

[10] J. Brooke, "SUS: A 'quick and dirty' usability scale," in *Usability Evaluation in Industry*, 1996, pp. 189–194.

Web references accessed November 2026. Detailed test records: `docs/checkpoints/` in the repository.

AI-use declaration. The team defined the product, made the design decisions and reviewed all changes. AI tools assisted with proposing implementation code, tests, UI refinements and review fixes, with drafting documentation and this report, and with language refinement and LaTeX preparation. All AI-proposed changes were checked against the source, contracts and test results; AI summaries were not treated as measured results.

TODO (team): Confirm this declaration matches what each member actually did, and add the team's own statement of what it learned from using AI (required by the module's plagiarism policy).